

TAKE-OFF WEIGHT PREDICTION

Sam Jones

1 Overview

For this project, I chose Python as my analysis tool due to my familiarity with essential libraries like Pandas, Scikit-Learn, and Plotly, which facilitated efficient data handling, visualisation, and modelling. While I initially considered experimenting with multiple models, I found the evaluation scores of the linear regression model satisfactory and, therefore, selected it as my final model.

2 Exploratory Data Analysis (EDA)

2.1 Dataset Overview

I began by analysing the shape, structure, missing values and datatypes of both datasets, discovering that the training set and validation set contained 29731 and 1878 rows respectively. Both contained an identical list of columns, aside from the target variable, *ActualTOW*, which appeared only in the training set.

Upon review of the datasets, I decided to retain the following columns for the analysis: *ActualFlightTime*, *ActualTotalFuel*, *FlownPassengers*, *BagsCount*, *FlightBagsWeight*, & *ActualTOW* (Target Variable). I selected these variables (**X**) for regression because they have a clear and logical impact on the target variable, *ActualTOW* (**y**).

The remaining columns, (*DepartureDate*, *DepartureYear*, *DepartureMonth*, *DepartureDay*, *FlightNumber*, *DepartureAirport*, *ArrivalAirport* & *Route*), were excluded for the regression, as they do not exhibit an obvious or meaningful relationship with take-off weight. However, the information contained in these was useful when logic-checking some results and outliers later in the process.

2.2 Data Cleaning & Preprocessing

For the overview of the 2 respective datasets, I decided to create functions in Python to avoid duplication of code. These functions covered data loading, initial exploratory analysis, dealing with null values, transforming data types, and dealing with outliers. These are detailed further below.

2.2.1 Missing Values

The first task when addressing missing values was to convert the values marked as (null) in string format to actual Python NoneTypes, which, in both the training and validation sets, revealed about 7-8% of missing data in the *BagsCount* and *FlightBagsWeight* columns. As these columns were highly correlated (>99%), I calculated the average bag weight and used that to fill missing data from one column to another, and vice versa, which dropped the null count to around 7% in each. For the remaining null values, I decided to use the KNN imputation technique to fill the values, as it considers the similarity between data points, in order to preserve the underlying patterns and relationships in the dataset.

2.2.2 Data Types

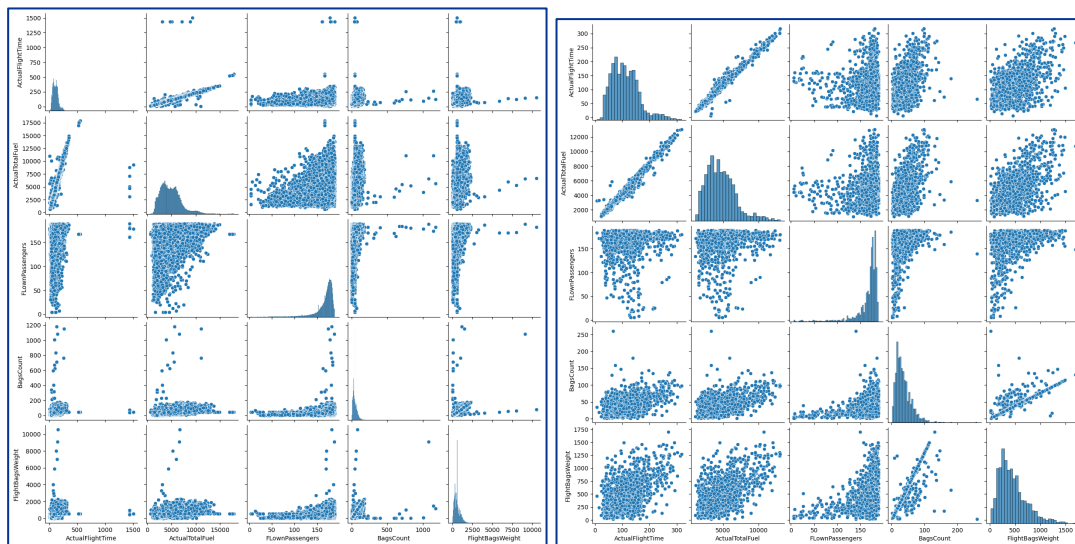
In both original datasets, due to the '(null)' string values, 4 of the columns were classified as object type. After transforming the null values, I was able to convert these columns to integer data types, which was necessary for the KNN imputation, the outlier detection and the regression itself.

2.2.3 Inconsistencies and Outliers

Through my analysis, I noticed some strange values on the upper end of some variables, which contradicted the prior knowledge that I had of the airline's service offering. Examples of this include [ActualFlightTime](#) values of over 1500 minutes (25 hours) which I concluded were due to flight layovers or delays appearing in the dataset. There were also some values around the 500-minute (8.33 hours) mark, which I was able to attribute as aircraft deliveries from Boeing HQ in Seattle to Dublin. There were also some extreme outliers in the [BagsCount](#) and [FlightBagsWeight](#) columns, which I was able to discard, having calculated that the max luggage capacity of Boeing 737-MAX 200, the airline's largest plane, is roughly 500, meaning all values above that were likely errors.

In the training set, I decided to remove these outliers to ensure that the model was not affected by the skewed values. While initially, I set parameters for max acceptable values as per my research above, but eventually, in the interest of consistency, I decided to use the ± 1.5 IQR method (discarding all values above 1.5 IQRs below and above the 1st and 3rd quartile respectively).

The pairwise scatter plots for the training set variables, used to initially identify outliers, can be seen below, before (left) and after (right) data cleaning and removal of outliers.



2.2.4 Duplicate Values

Initially, I detected no explicit duplicate values in my EDA. However, during later stages of my analysis, I noticed certain number of flights which were on the same day and shared some characteristics which made them highly unlikely to be different flights. As such, I decided to drop these from the training set to avoid skewing of the model. My methodology of choice for this was to identify duplicates based on a subset of columns. I chose to remove all rows that were identical in [DepartureDate](#), [DepartureAirport](#) and [ArrivalAirport](#) as well as 1 of

ActualFlightTime, *ActualTotalFuel*, *FlownPassengers*, *FlightBagsWeight* or *BagsCount*. While I accept that this methodology is not perfect, I see it as an efficient way to remove entries that are, on the balance of probability, likely duplicate values.

3 Model Development

3.1 Feature Selection

From the remaining variables, I decided to carry out the Stepwise selection method to determine which would be most beneficial to include in the model. the results of the process were as follows:

- *ActualFlightTime* with p-value **0.0**
- *FlownPassengers* with p-value **0.0**
- *ActualTotalFuel* with p-value **0.0**
- *FlightBagsWeight* with p-value **2.408e-62**

As such, the *BagsCount* variable was discarded in this step. To improve the model results further and detect any potential Multicollinearity amongst the remaining explanatory variables, I conducted a VIF analysis,

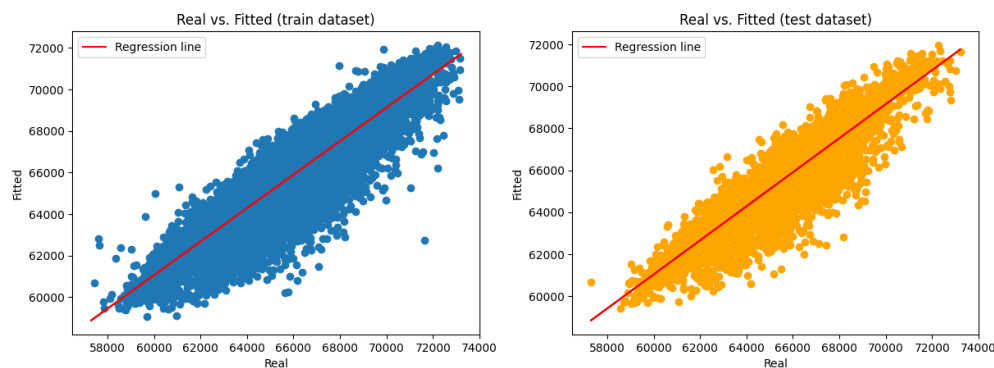
3.2 Training Process

I decided to split the training data 80%-20% to allow me to test my model before applying it to the validation set. This left me with a train dataset of 19742 rows and test dataset of 4936 rows.

The model was then trained on the first set and tested on the second set.

3.3 Evaluation

The real vs. fitted data was plotted on the charts below to give an indication of the accuracy of the model.



The error values of the model were as follows:

	<i>Train</i>	<i>Test</i>
R Squared	0.81	0.82
Root Mean Square Error	1097.21	1088.89